

ML From Scratch

A 12-Week, Day-by-Day Project Guideline

From linear regression to a Transformer, built with your own hands

Built for: MSc scholarship portfolio (abroad)

Level: CSE 9th semester, theory done, library fluency to build

Pace: 1 to 2 hours per day, around coursework

Phase	What you build	Weeks
1	Linear and logistic regression, plus all evaluation metrics and plots	1 - 2
2	Your own autograd engine (a mini PyTorch core)	3 - 4
3	Neural network (MLP) and optimizers on MNIST, in pure numpy	5 - 6
4	Paper-to-code: Transformer attention from scratch, plus a small language model	7 - 9
5	PyTorch fluency rebuild, then polish and portfolio	10 - 12

1. Read this first: how this guide works

Your situation in one line: you know the theory and the math, but library code does not stay in your head, and past projects leaned heavily on AI-generated code. This plan fixes exactly that gap by making you **write every important line yourself**, starting from bare numpy and ending in PyTorch. By the end you will have five small, complete, shippable projects. Each one is a story you can tell in an SOP or interview.

The one rule that makes it work

Attempt first, look up second, retype third. For every task: (1) try for 15 to 20 minutes with nothing open except docs. (2) If stuck, search or ask AI for a *hint or explanation, not the full code*. (3) Close the answer and type the solution from your own understanding. (4) Write the pattern in your cheat-sheet. If you paste code you do not understand, the day does not count.

How to use AI without losing the learning

- Allowed: ask it to explain an error message, explain a concept differently, review your finished code, or suggest test cases.
- Allowed: ask for a tiny hint such as "which numpy function reshapes wide to long?" and then write the line yourself.
- Not allowed during the attempt phase: asking it to write the whole function or the whole training loop.
- Best habit: after finishing a day, paste your code and ask "what bugs or bad practices do you see?". You learn from the critique.

Time structure of one normal day (60 to 120 minutes)

Block	Minutes	What happens
Warm-up recall	10	Rewrite from memory yesterday's key snippet (5 to 10 lines) without looking. This is the spaced-retrieval trick that moves things from recognized to recalled.
Main work	40 - 80	The day's task from the weekly table. Attempt first, then look up, then retype.
Cheat-sheet + commit	10	Add 1 to 3 patterns to cheat_sheet.md, then git commit with a clear message.
Shutdown note	2	One line in log.md: what worked, what blocked you, first task for tomorrow.

Weekly rhythm

- **Day 1 to Day 5:** learning and building.
- **Day 6:** consolidation: clean the code, write tests, plot results, update README.
- **Day 7:** light review or full rest. Never skip rest for more than two weeks in a row; this is a marathon of about 12 weeks.
- If a week goes badly because of exams or assignments, do only Day 1 to Day 3 tasks and keep the streak alive. Do not try to catch up all at once.

2. Setup (do this once, about 1 to 2 hours)

Tools

Tool	Why	Command / note
Python 3.10+	Everything runs here	Check with <code>python --version</code>
Virtual environment	Keeps libraries clean and reproducible	<code>python -m venv .venv</code> , then activate it
VS Code + Jupyter	Notebooks for exploring, .py files for final code	Install the Python and Jupyter extensions
Git + GitHub	Public proof of your work; commit daily	<code>git init</code> , create a GitHub repo named <code>ml-from-scratch</code>
Libraries	Core stack	numpy, pandas, matplotlib, scikit-learn (only for checking), pytest, jupyter; PyTorch from week 10
Google Colab / Kaggle	Free GPU for weeks 8 to 11 if your laptop is slow	Keep code in GitHub so you can pull it in Colab

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install numpy pandas matplotlib scikit-learn pytest jupyter
pip freeze > requirements.txt
```

Repository structure (one repo, five projects)

```
ml-from-scratch/
|-- README.md                <- the portfolio front page (updated every phase)
|-- requirements.txt
|-- cheat_sheet.md           <- your personal pattern notebook (most important file)
|-- log.md                   <- one line per day
|-- 01_regression/
|   |-- linear_regression.py
|   |-- logistic_regression.py
|   |-- metrics.py
|   |-- notebooks/
|   |-- results/             <- saved plots
|   `-- tests/
|-- 02_autograd/
|   |-- engine.py            <- Value class
|   |-- nn.py                <- Neuron, Layer, MLP built on engine
|   `-- tests/
|-- 03_mlp_numpy/
|   |-- mlp.py
|   |-- optimizers.py
|   |-- train_mnist.py
|-- 04_transformer/
|   |-- attention.py
|   |-- transformer.py
|   |-- train_lm.py
|   |-- ablations/
|-- 05_pytorch/
|   |-- rebuilds of every earlier project in PyTorch
|-- report/
|   |-- final_report.md      <- mini-paper style write-up
```

Datasets you will use

Phase	Dataset	How to get it
1	California Housing (regression), Breast Cancer Wisconsin (classification)	sklearn.datasets, no download needed
2	Tiny toy data (moons or a hand-made curve)	sklearn.datasets.make_moons
3	MNIST handwritten digits	torchvision or keras datasets, or a CSV from Kaggle; load once and save as .npz
4	Tiny Shakespeare text (char-level) or a small Bangla text corpus	Plain .txt file; Bangla corpus is a great local-relevance angle
5	Reuse all of the above	Same data, PyTorch code

3. Roadmap at a glance

Week	Topic	Main output
1	numpy and pandas foundations, linear regression (closed form + gradient descent)	Working linear regression with loss curve
2	Logistic regression and every evaluation metric	Metrics module: confusion matrix, precision, recall, F1, ROC, PR curve
3	Autograd core: Value class, add, mul, backward	Engine that matches hand-computed gradients
4	Autograd complete plus Neuron, Layer, MLP on top	Small net that learns the moons dataset
5	Numpy MLP with manual backprop on MNIST	Around 95 percent test accuracy
6	Optimizers (SGD, momentum, Adam), regularization, experiments	97 percent or more, comparison plots
7	Paper-to-code part 1: read the Transformer paper, build attention	Scaled dot-product and multi-head attention in numpy
8	Paper-to-code part 2: full Transformer block and a small language model	Model that generates text
9	Paper-to-code part 3: debugging, ablations, analysis	Ablation table and short analysis
10	PyTorch fluency: tensors, autograd, nn.Module, training loop	Earlier MLP rebuilt in PyTorch
11	PyTorch rebuild of the Transformer and of a piece of your research-paper code	Side-by-side comparison with your numpy version
12	Polish: README, report, plots, demo, SOP paragraph	Portfolio-ready repository and report

Checkpoint principle: every phase ends with something complete. If you only finish Phase 1 to 3 before an application deadline, you still have a strong, honest portfolio. Phase 4 (the paper-to-code work) is the part admissions committees remember most, so protect its time.

Week 1: Foundations and Linear Regression

Phase: 1: Regression and metrics | **Time:** 1-2 hrs/day

Goal of the week: Get comfortable with numpy and pandas by building linear regression twice: once with the closed-form (normal equation) and once with gradient descent.

Day	What to do	How to do it (steps)	Output of the day
Day 1	Setup and numpy warm-up	Finish Section 2 setup. Then in a notebook practice: array creation, shapes, reshape, broadcasting, slicing, axis-wise sum and mean, matrix multiply with @. Predict each output shape BEFORE running.	Notebook numpy_warmup.ipynb; first cheat-sheet entries
Day 2	pandas warm-up on real data	Load California Housing into a DataFrame. Practice head, describe, isna, groupby, filter with boolean masks, sort_values, merge. Plot 3 histograms and 1 scatter with matplotlib.	EDA notebook with 4 plots
Day 3	Data preparation	Split into train and test yourself (shuffle with a fixed seed, no sklearn). Standardize features using train mean and std only. Add a bias column. Explain in a markdown cell why test data must not leak into scaling.	prepare_data() function
Day 4	Closed-form linear regression	Implement $w = \text{solve}(X.T @ X, X.T @ y)$. Prefer <code>np.linalg.solve</code> over inverse and note why. Compute predictions and MSE on train and test. Compare with sklearn <code>LinearRegression</code> to verify.	linear_regression.py with <code>fit_closed_form()</code>
Day 5	Gradient descent version	Write the MSE gradient by hand from your math knowledge. Loop: predict, compute loss, compute gradients, update weights. Record loss each step and plot the curve. Try three learning rates.	Loss curves for 3 learning rates
Day 6	Consolidate	Wrap both methods in a <code>LinearRegression</code> class with <code>fit</code> , <code>predict</code> , <code>score</code> . Add 3 pytest tests (perfect line data gives near-zero loss, shapes are correct, GD result close to closed-form).	Class + passing tests; README section
Day 7	Rest / light review	Rewrite the gradient descent loop from memory (no peeking). Note what you forgot in <code>cheat_sheet.md</code> .	Cheat-sheet updated

Starter pattern for this week (read it, then close it and write it yourself)

```
import numpy as np

def predict(X, w):
    # X already has a bias column of ones
    return X @ w

def mse(y, y_hat):
    return np.mean((y - y_hat) ** 2)

def gradient(X, y, w):
    n = len(y)
    return (2 / n) * X.T @ (X @ w - y)

# training loop
w = np.zeros(X.shape[1])
for step in range(1000):
    w -= lr * gradient(X, y, w)
```

Tip: If the loss goes to infinity, your learning rate is too big or your features are not standardized. This bug teaches more than any lecture.

Weekly deliverable

01_regression/ contains a clean LinearRegression class (closed form and gradient descent), a notebook that shows the loss curve, and a README paragraph with your test MSE and R-squared.

You are done with this week when

- Your gradient descent result matches closed-form within a small tolerance.
- You can explain, without notes, why we standardize features and what the learning rate does.
- You can write the gradient of MSE on paper in under a minute.

Week 2: Logistic Regression and Evaluation Metrics

Phase: 1: Regression and metrics | **Time:** 1-2 hrs/day

Goal of the week: Build logistic regression for classification and implement every evaluation metric yourself, so TP, FP, precision, recall and ROC stop being just formulas.

Day	What to do	How to do it (steps)	Output of the day
Day 1	Sigmoid and the loss	Implement sigmoid (stable version) and binary cross-entropy. Test with extreme inputs like -1000 and 1000 and fix any overflow or log(0) problems using clipping.	sigmoid() and bce_loss() with tests
Day 2	Gradient and training	Derive the gradient (it simplifies to $X.T @ (p - y) / n$). Implement the training loop on Breast Cancer data (standardized). Plot loss per step.	Trained model, loss curve
Day 3	Confusion matrix and basic metrics	In metrics.py compute TP, FP, TN, FN with boolean masks. Then accuracy, precision, recall, specificity, F1. Verify each against sklearn.metrics.	metrics.py part 1 + comparison test
Day 4	Threshold and curves	Loop over thresholds 0 to 1. Build ROC (TPR vs FPR) and precision-recall curves. Compute AUC with the trapezoid rule (np.trapz or np.trapezoid).	ROC and PR plots from scratch
Day 5	Imbalanced data experiment	Artificially drop most positive samples. Show that accuracy stays high while recall collapses. Try class weights in the loss. Plot before and after.	Short written finding with 2 plots
Day 6	Consolidate	Add regularization (L2) to logistic regression and compare. Write a clean report notebook: data, method, results table, plots. Update README.	Phase 1 mini-project complete
Day 7	Rest / review	Write the confusion matrix formulas from memory. Explain precision vs recall to an imaginary friend in 5 sentences.	Cheat-sheet updated

Starter pattern for this week (read it, then close it and write it yourself)

```
def confusion(y_true, y_pred):
    tp = np.sum((y_true == 1) & (y_pred == 1))
    fp = np.sum((y_true == 0) & (y_pred == 1))
    tn = np.sum((y_true == 0) & (y_pred == 0))
    fn = np.sum((y_true == 1) & (y_pred == 0))
    return tp, fp, tn, fn

def precision_recall_f1(tp, fp, fn, eps=1e-12):
    precision = tp / (tp + fp + eps)
    recall = tp / (tp + fn + eps)
    f1 = 2 * precision * recall / (precision + recall + eps)
    return precision, recall, f1
```

Tip: Connect this to your research: security datasets are usually heavily imbalanced, so Day 5 is directly useful later.

Weekly deliverable

Phase 1 is now a complete mini-project: regression and classification from scratch, a metrics module, ROC and PR curves, and one experiment on imbalance. You could already show this.

You are done with this week when

- Your metrics match sklearn on at least 3 random test cases.
- You can explain when to prefer recall over precision using a real example (for example intrusion detection).
- Phase 1 folder has tests and a README with result plots.

Week 3: Autograd Engine, Part 1

Phase: 2: Autograd | Time: 1-2 hrs/day

Goal of the week: Understand backpropagation as a graph algorithm by building a tiny scalar autograd engine (in the spirit of Andrej Karpathy's micrograd).

Day	What to do	How to do it (steps)	Output of the day
Day 1	Derivatives by hand	On paper, compute gradients for $f = (a*b + c)**2$ using the chain rule. Draw the computation graph with nodes and arrows. Do 3 more examples with different operations.	Paper notes (photo it into the repo)
Day 2	Value class skeleton	Create class Value with data, grad, _prev (children), _op, and a _backward function. Implement __repr__ and __add__ with its backward step (gradient flows to both inputs, using += not =).	engine.py with add working
Day 3	Multiplication and friends	Add __mul__, __neg__, __sub__, __radd__, __rmul__. Remember: $d(a*b)/da = b$, $d(a*b)/db = a$. Test that $a + 1$ and $2 * a$ both work.	Add, mul, sub, neg all pass tests
Day 4	The backward() method	Build a topological ordering of the graph with a DFS. Set the output grad to 1, then call each node's _backward in reverse order. Explain in a comment why order matters.	backward() working on small graphs
Day 5	Verify against PyTorch or numeric gradients	Compare your gradients with a numerical estimate $(f(x+h) - f(x-h)) / 2h$. If PyTorch is installed, compare with it too. Fix every mismatch.	test_engine.py with gradient checks
Day 6	Visualize	Optionally draw the computation graph (graphviz or just print it as text). Write a short markdown explanation of backprop in your own words with a diagram.	Explanation section in README
Day 7	Rest / review	Re-implement __mul__ and backward from memory. Note what you missed.	Cheat-sheet updated

Starter pattern for this week (read it, then close it and write it yourself)

```
class Value:
    def __init__(self, data, _children=(), _op=""):
        self.data = data
        self.grad = 0.0
        self._backward = lambda: None
        self._prev = set(_children)
        self._op = _op

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data + other.data, (self, other), "+")
        def _backward():
            self.grad += out.grad
            other.grad += out.grad
        out._backward = _backward
        return out
```

Tip: The single most common bug is writing `self.grad = ...` instead of `self.grad += ....` If a variable is used twice, the second use overwrites the first.

Weekly deliverable

02_autograd/engine.py with a Value class supporting add, mul, sub, neg and backward(), plus a test file proving gradient correctness.

You are done with this week when

- Gradients match numerical gradients within $1e-6$ on at least 5 expressions.
- You can explain why gradients are accumulated with += (a variable used twice).
- You can draw the graph of any small expression and run backprop on it by hand.

Week 4: Autograd Engine, Part 2, and a Neural Net on Top

Phase: 2: Autograd | Time: 1-2 hrs/day

Goal of the week: Finish the engine with power, division, tanh, relu and exp, then build Neuron, Layer and MLP classes that train on a real (small) dataset.

Day	What to do	How to do it (steps)	Output of the day
Day 1	More operations	Add <code>__pow__</code> (constant exponent), <code>__truediv__</code> , <code>exp</code> , <code>tanh</code> , <code>relu</code> . Derive each local derivative on paper first, then code it.	engine.py with 5 more ops
Day 2	Gradient-check everything	Extend your test file: every operation gets a numerical gradient check. Add a test for a long expression that uses the same Value several times.	All tests green
Day 3	Neuron and Layer	In <code>nn.py</code> : Neuron holds weights (random small Values) and a bias; <code>__call__</code> computes activation($\sum(w*x) + b$). Layer is a list of Neurons. Add <code>parameters()</code> methods.	Neuron and Layer classes
Day 4	MLP class	MLP takes sizes like <code>[2, 8, 8, 1]</code> and chains Layers. Write <code>zero_grad()</code> and a simple SGD update: <code>p.data -= lr * p.grad</code> .	MLP forward pass works
Day 5	Train on moons	Generate <code>make_moons</code> data. Use a loss such as hinge or MSE on scalar outputs. Loop: forward, <code>zero_grad</code> , backward, update. Log loss every 10 steps.	Loss decreasing, accuracy above 90 percent
Day 6	Decision boundary plot	Evaluate your MLP over a grid and plot the boundary with <code>matplotlib</code> (contourf). Save the figure. Write the Phase 2 README section.	Decision-boundary image in results/
Day 7	Rest / review	From memory: write the <code>tanh</code> op with its backward. Explain why <code>relu</code> is easy to differentiate.	Cheat-sheet updated

Starter pattern for this week (read it, then close it and write it yourself)

```
def tanh(self):
    import math
    t = math.tanh(self.data)
    out = Value(t, (self,), "tanh")
    def _backward():
        self.grad += (1 - t ** 2) * out.grad
    out._backward = _backward
    return out

# training loop
for k in range(100):
    preds = [model(x) for x in Xs]
    loss = sum((p - y) ** 2 for p, y in zip(preds, ys))
    model.zero_grad()
    loss.backward()
    for p in model.parameters():
        p.data -= lr * p.grad
```

Tip: Scalar autograd is slow. That is fine and intentional: it shows you exactly what tensors will later do in bulk. Use small data (about 100 points).

Weekly deliverable

Phase 2 complete: a working scalar autograd engine and a small neural net trained with it, with tests and a decision-boundary figure.

You are done with this week when

- Your MLP learns moons to above 90 percent accuracy using only your own engine.
- You can explain forward pass, backward pass, and parameter update in plain words.
- Tests cover every operation.

Week 5: Numpy MLP with Manual Backprop on MNIST

Phase: 3: Neural network | **Time:** 1-2 hrs/day

Goal of the week: Move from scalars to matrices: build a two-layer network in pure numpy, derive the matrix gradients yourself, and train it on MNIST.

Day	What to do	How to do it (steps)	Output of the day
Day 1	Load and inspect MNIST	Load images, flatten 28x28 to 784, scale to 0-1, one-hot encode labels. Show 10 sample images with matplotlib. Save as .npz for fast reloads.	Data loader function
Day 2	Forward pass	Initialize weights (He init: $\text{randn} * \sqrt{2/\text{fan_in}}$). Implement affine layer, ReLU, and a numerically stable softmax (subtract the row max). Compute cross-entropy loss.	forward() returns loss and cache
Day 3	Backward pass on paper, then code	Derive $\text{dz2} = (\text{probs} - \text{y_onehot}) / N$, then dW2 , db2 , da1 , dz1 , dW1 , db1 . Write them down first, then code them. Check shapes at every line.	backward() written
Day 4	Gradient check	Compare your analytic gradients with numerical gradients on a tiny network (about 5 samples, 10 hidden units). Relative error should be below $1e-5$. Fix any mismatch.	Gradient-check test passing
Day 5	Training loop with mini-batches	Shuffle each epoch, iterate mini-batches of 64 or 128, update with SGD. Log train and validation loss and accuracy per epoch. Plot them.	First MNIST training run
Day 6	Consolidate	Turn the code into a clean MLP class (fit, predict, evaluate). Save a confusion matrix using your Phase 1 metrics module. Write README section.	About 95 percent test accuracy
Day 7	Rest / review	Write the backward pass equations from memory on paper.	Cheat-sheet updated

Starter pattern for this week (read it, then close it and write it yourself)

```
z1 = X @ W1 + b1
a1 = np.maximum(0, z1)
z2 = a1 @ W2 + b2
shifted = z2 - z2.max(axis=1, keepdims=True)
probs = np.exp(shifted) / np.exp(shifted).sum(axis=1, keepdims=True)

dz2 = (probs - Y_onehot) / N
dW2 = a1.T @ dz2
db2 = dz2.sum(axis=0)
da1 = dz2 @ W2.T
dz1 = da1 * (z1 > 0)
dW1 = X.T @ dz1
db1 = dz1.sum(axis=0)
```

Tip: When shapes mismatch, print every shape. Almost every backprop bug is a transposed matrix.

Weekly deliverable

03_mlp_numpy/ with a two-layer numpy MLP, a gradient-check test, training curves, and a confusion matrix on MNIST.

You are done with this week when

- Numeric and analytic gradients agree.
- Test accuracy is around 95 percent or higher.
- You can explain every shape in the backward pass.

Week 6: Optimizers, Regularization and Experiments

Phase: 3: Neural network | Time: 1-2 hrs/day

Goal of the week: Implement SGD with momentum and Adam from their formulas, add regularization, and run clean experiments. This is where your work starts to look like research.

Day	What to do	How to do it (steps)	Output of the day
Day 1	SGD with momentum	Implement velocity $v = \text{beta} \cdot v + \text{grad}$, then $w -= \text{lr} \cdot v$. Compare against plain SGD over 10 epochs. Plot both loss curves on one chart.	optimizers.py with SGD and Momentum
Day 2	Adam	Implement m , v , bias correction and the update step from the paper's formulas. Test on the same run. Keep your hyperparameters in a dictionary.	Adam optimizer working
Day 3	Regularization	Add L2 weight decay and dropout (with correct scaling at train time). Compare validation curves with and without.	Overfitting experiment plot
Day 4	Learning rate study	Sweep learning rates on a log scale ($1e-4$ to 1). Plot final validation accuracy vs learning rate for each optimizer.	LR sensitivity figure
Day 5	Initialization and depth	Compare zero, small random, and He initialization. Try 1, 2, and 3 hidden layers. Record results in a table.	Results table in CSV/markdown
Day 6	Write-up	Write a 1-page experiment report: setup, results table, 3 plots, 3 conclusions. Aim for 97 percent or more with your best setup. Update README.	Phase 3 report
Day 7	Rest / review	Write the Adam update from memory.	Cheat-sheet updated

Starter pattern for this week (read it, then close it and write it yourself)

```
class Adam:
    def __init__(self, params, lr=1e-3, b1=0.9, b2=0.999, eps=1e-8):
        self.params, self.lr, self.b1, self.b2, self.eps = params, lr, b1, b2, eps
        self.m = [np.zeros_like(p) for p in params]
        self.v = [np.zeros_like(p) for p in params]
        self.t = 0

    def step(self, grads):
        self.t += 1
        for i, (p, g) in enumerate(zip(self.params, grads)):
            self.m[i] = self.b1 * self.m[i] + (1 - self.b1) * g
            self.v[i] = self.b2 * self.v[i] + (1 - self.b2) * g ** 2
            m_hat = self.m[i] / (1 - self.b1 ** self.t)
            v_hat = self.v[i] / (1 - self.b2 ** self.t)
            p -= self.lr * m_hat / (np.sqrt(v_hat) + self.eps)
```

Tip: Fix random seeds and run each experiment at least 3 times. Reporting mean and standard deviation is what makes it feel like research.

Weekly deliverable

Phase 3 complete: an MNIST network with three optimizers, regularization, learning-rate and initialization studies, and a written experiment report.

You are done with this week when

- Best model reaches about 97 percent or more.
- You can explain what momentum and Adam fix compared with plain SGD.
- Every plot has axis labels, a title, and a legend.

Week 7: Paper-to-Code, Part 1: Reading the Paper and Building Attention

Phase: 4: Paper to code | **Time:** 1-2 hrs/day (aim for the upper end)

Goal of the week: Read "Attention Is All You Need" properly, and implement scaled dot-product attention and multi-head attention in numpy.

Day	What to do	How to do it (steps)	Output of the day
Day 1	First read	Read the abstract, introduction, and Figure 1 and 2. Do not try to understand everything. Write down 5 questions you have. Watch or read one visual explainer of self-attention for intuition.	questions.md with 5 questions
Day 2	Second read, equations	Read Section 3 slowly. For every equation write the tensor shapes next to it (batch, heads, sequence, dimension). Redraw Figure 2 by hand.	Shape notes page
Day 3	Scaled dot-product attention	Implement $\text{softmax}(Q K^T / \sqrt{d_k}) V$ in numpy. Test with tiny inputs where you can compute the answer by hand. Explain why we divide by $\sqrt{d_k}$.	attention.py: scaled_dot_product()
Day 4	Masking	Add a causal mask (upper triangle set to a large negative number before softmax). Verify that position i cannot attend to j greater than i by printing the attention matrix.	Masked attention verified
Day 5	Multi-head attention	Split d_{model} into heads, apply attention per head, concatenate, project. Use reshape and transpose carefully and check shapes with asserts.	multi_head_attention() with shape asserts
Day 6	Positional encoding + visualization	Implement sinusoidal positional encoding. Plot it as a heatmap. Plot an attention matrix for a short toy sentence.	Two plots in results/
Day 7	Rest / review	Explain self-attention out loud in 2 minutes without notes.	Cheat-sheet updated

Starter pattern for this week (read it, then close it and write it yourself)

```
def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)
    e = np.exp(x)
    return e / e.sum(axis=axis, keepdims=True)

def scaled_dot_product(Q, K, V, mask=None):
    d_k = Q.shape[-1]
    scores = Q @ K.swapaxes(-1, -2) / np.sqrt(d_k)    # (... , T, T)
    if mask is not None:
        scores = np.where(mask, scores, -1e9)
    weights = softmax(scores, axis=-1)
    return weights @ V, weights
```

Tip: This is the hardest phase and the most valuable. If a day takes two days, that is normal. Budget for it; week 11 has spare time for catch-up.

Weekly deliverable

04_transformer/attention.py with tested scaled dot-product attention, causal masking, multi-head attention, and positional encoding, plus your notes on the paper.

You are done with this week when

- You can state the shape of Q, K, V, scores and output at every step.
- Masked attention matrix is lower-triangular after softmax.
- You can explain why multi-head attention helps.

Week 8: Paper-to-Code, Part 2: Transformer Block and a Small Language Model

Phase: 4: Paper to code | **Time:** 1-2 hrs/day (aim for the upper end)

Goal of the week: Assemble a full Transformer decoder block and train a small character-level language model. You will switch to PyTorch for training this week because training in numpy would need manual backprop through attention. Keep the model code written by you.

Day	What to do	How to do it (steps)	Output of the day
Day 1	PyTorch crash-lesson (only what you need)	Learn tensors, requires_grad, nn.Module, nn.Linear, nn.Embedding, optimizers, and the training loop. Re-implement your MLP forward pass as an nn.Module as a warm-up.	Small MLP in PyTorch
Day 2	Port attention to PyTorch	Rewrite scaled dot-product and multi-head attention as an nn.Module. Test that the outputs match your numpy version on the same random input (use np.allclose).	Numpy vs PyTorch equality test
Day 3	Feed-forward, LayerNorm, residuals	Implement the position-wise feed-forward network, use nn.LayerNorm, and wire residual connections. Choose pre-norm (simpler to train) and note the difference from the paper.	TransformerBlock class
Day 4	Full decoder model	Add token embedding, positional embedding, N stacked blocks, final LayerNorm and a linear head to vocabulary size. Count parameters.	GPT-style model class; parameter count
Day 5	Data and training loop	Build a character vocabulary from tiny Shakespeare (or Bangla text). Create random (input, target shifted by 1) batches. Train with AdamW; log train and validation loss.	First training run
Day 6	Generation	Write a sample() function using temperature and top-k. Generate 500 characters. Save outputs at step 0, mid, and end of training to show progress.	Generated samples file
Day 7	Rest / review	Sketch the whole model architecture from memory as a block diagram.	Cheat-sheet updated

Starter pattern for this week (read it, then close it and write it yourself)

```
class TransformerBlock(nn.Module):
    def __init__(self, d_model, n_heads, d_ff, dropout=0.1):
        super().__init__()
        self.ln1 = nn.LayerNorm(d_model)
        self.attn = MultiHeadAttention(d_model, n_heads)
        self.ln2 = nn.LayerNorm(d_model)
        self.ff = nn.Sequential(nn.Linear(d_model, d_ff), nn.GELU(),
                                nn.Linear(d_ff, d_model), nn.Dropout(dropout))

    def forward(self, x, mask):
        x = x + self.attn(self.ln1(x), mask)
        x = x + self.ff(self.ln2(x))
        return x
```

Tip: Use a small model (for example 4 layers, d_model 128, 4 heads) so training fits on Colab or even a laptop CPU. The goal is understanding, not size.

Weekly deliverable

A working small Transformer language model that you wrote yourself, with training curves and generated text samples showing quality improving over training.

You are done with this week when

- Loss decreases clearly, and generated text starts to look like words.
- You can label every block on a diagram of your model.
- Numpy and PyTorch attention outputs match.

Week 9: Paper-to-Code, Part 3: Debugging, Ablations and Analysis

Phase: 4: Paper to code | **Time:** 1-2 hrs/day

Goal of the week: Turn the working model into research: run controlled ablations, analyze results, and write up what you learned. This is the part that impresses reviewers.

Day	What to do	How to do it (steps)	Output of the day
Day 1	Experiment design	Decide 4 ablations: (a) no positional encoding, (b) 1 head vs 4 heads, (c) no residual connections, (d) different model depth. Write the hypothesis for each BEFORE running.	ablation_plan.md with hypotheses
Day 2	Run ablations (a) and (b)	Same seed, same data, same steps. Save the validation loss curve and final loss for each run into a CSV.	Results CSV rows
Day 3	Run ablations (c) and (d)	Same protocol. Keep an experiment log with the config for every run so any result can be reproduced.	Results CSV complete
Day 4	Analysis plots	Plot all runs on shared axes. Visualize attention maps from 2 heads on a sample text and describe what patterns you see.	Comparison figure + attention maps
Day 5	Write the mini-paper	Write 2 to 3 pages: Introduction, Method, Experiments, Results, Limitations. Compare each result with your hypothesis (including where you were wrong).	report/transformer_report.md
Day 6	Cleanup and reproducibility	Add a single command to reproduce the main result. Pin library versions. Add a README with the results table.	Reproducible repo section
Day 7	Rest / buffer	Use this day as buffer if Phase 4 slipped. Otherwise rest.	-

Tip: Negative or surprising results are good. Explaining why an ablation did not behave as expected shows real understanding.

Weekly deliverable

A short research-style report with an ablation table, plots, attention visualizations, and an honest limitations section.

You are done with this week when

- Each ablation has a hypothesis, result and one-sentence explanation.
- Someone else can reproduce your main result from the README alone.
- You can explain the paper without looking at it.

Week 10: PyTorch Fluency, Part 1

Phase: 5: PyTorch rebuild | **Time:** 1-2 hrs/day

Goal of the week: Now that you know what happens under the hood, rebuild your earlier projects in idiomatic PyTorch. Because you wrote the internals yourself, everything will feel familiar rather than magical.

Day	What to do	How to do it (steps)	Output of the day
Day 1	Tensors and autograd	Re-create your Week 3 tests using torch tensors with <code>requires_grad=True</code> . Compare torch's gradients to your engine's. Practice reshape, view, transpose, cat, stack, indexing.	Notebook comparing your engine to torch
Day 2	nn.Module and loss functions	Rebuild the MNIST MLP as an nn.Module with nn.Linear and nn.ReLU. Use nn.CrossEntropyLoss (note that it expects raw logits, not softmax).	MLP module
Day 3	Dataset and DataLoader	Write a custom Dataset class for MNIST. Use DataLoader with batch_size, shuffle, and a train/validation split via random_split.	Data pipeline
Day 4	The standard training loop	Write train_one_epoch() and evaluate() functions with model.train(), model.eval(), torch.no_grad(), optimizer.zero_grad(), loss.backward(), optimizer.step(). Learn it until you can type it blind.	Reusable train/eval functions
Day 5	Checkpoints, GPU, schedulers	Save and load the best model with state_dict. Move model and data to the device. Add a learning-rate scheduler and early stopping.	Best-checkpoint saving
Day 6	Compare	Compare accuracy, code length, and speed of your numpy MLP versus PyTorch. Write down what PyTorch does for you automatically.	Comparison table in README
Day 7	Rest / review	Write the standard training loop from memory. This is the single most important snippet to know by heart.	Cheat-sheet updated

Starter pattern for this week (read it, then close it and write it yourself)

```
for epoch in range(epochs):
    model.train()
    for xb, yb in train_loader:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        loss = loss_fn(model(xb), yb)
        loss.backward()
        optimizer.step()

    model.eval()
    with torch.no_grad():
        val_acc = evaluate(model, val_loader)
```

Weekly deliverable

05_pytorch/ with an idiomatic PyTorch pipeline (Dataset, DataLoader, training loop, checkpoints) reproducing your MNIST results.

You are done with this week when

- You can write the PyTorch training loop from memory.
- PyTorch MLP accuracy is at least as good as your numpy version.
- You know what `zero_grad`, `backward` and `step` each do internally.

Week 11: PyTorch Fluency, Part 2: Rebuild Your Research Code

Phase: 5: PyTorch rebuild | **Time:** 1-2 hrs/day

Goal of the week: Apply your new fluency to the code that used AI heavily in the past. Rebuild one meaningful piece of your research-paper or earlier project code yourself, then compare. This closes your original gap. Days 6 and 7 double as catch-up buffer for any earlier week.

Day	What to do	How to do it (steps)	Output of the day
Day 1	Choose the target	Pick one component from your research or past project (for example the jailbreak classifier or the guardrail evaluation pipeline). Write what it should do, its inputs and outputs, and how you would test it.	target_spec.md
Day 2	Rebuild, first half	Write the data loading and preprocessing yourself using your cheat-sheet patterns. Look things up only after a real attempt.	Data pipeline rewritten
Day 3	Rebuild, second half	Write the model and the training or evaluation loop yourself. Use your metrics module from Week 2 for evaluation.	Full pipeline running
Day 4	Compare with the original	Compare results, code clarity, and bugs found. Fix issues in the original if you find any. Note every concept that was shaky.	comparison notes
Day 5	Reproduce the Transformer in PyTorch cleanly	Refactor your Week 8 model into clean modules with config dataclass, argparse, and a train.py script that runs from the command line.	python train.py --config works
Day 6	Buffer / catch-up	Finish anything unfinished from Weeks 7 to 10, or extend an ablation. Do not start anything new today.	Loose ends closed
Day 7	Rest	Take a full day off.	-

Tip: Optional extension: add a Bangla text or Bangla-security angle to the Transformer project (tokenization, small corpus). It ties your work to your own context and makes it memorable.

Weekly deliverable

A rebuilt, self-written version of one of your real research components, with a short comparison note, plus a command-line-runnable Transformer training script.

You are done with this week when

- You wrote the rebuilt component without pasting AI code.
- You can explain every line of it in an interview.
- The training script runs from a clean environment using only requirements.txt.

Week 12: Polish and Portfolio

Phase: 5: Portfolio | **Time:** 1-2 hrs/day

Goal of the week: Package the whole journey so a reviewer can understand it in 5 minutes and dive deep in 30. Presentation matters as much as code at this stage.

Day	What to do	How to do it (steps)	Output of the day
Day 1	Repository cleanup	Format code, remove dead code, add docstrings and type hints to key functions, ensure all tests pass with pytest. Add a .gitignore, a LICENSE, and pinned requirements.	Clean repo
Day 2	Main README	Write the front page: one-paragraph summary, table of the 5 projects with result numbers, key figures, how to run, folder map, and what you learned.	README.md finished
Day 3	Final report	Combine the phase reports into a single 6 to 10 page report_final.md (or PDF): motivation, methods, experiments, results, limitations, future work.	report/final_report.md
Day 4	Demo material	Create one notebook that runs the complete story in under 10 minutes on Colab. Record a 3 to 5 minute screen video walking through it.	Demo notebook + video link
Day 5	Scholarship framing	Write a 200-word SOP paragraph and a 3-bullet CV entry (see Section 4 for templates). Ask a professor or senior to read it and give feedback.	SOP paragraph + CV bullets
Day 6	Fresh-eyes test	Ask a friend to clone the repo and follow the README on their machine. Fix every place where they get stuck.	Reproducibility verified
Day 7	Celebrate and plan the next step	Choose your next project (for example extend the Zero Trust or guardrail research using the skills gained). Write 5 lines about it.	next_steps.md

Weekly deliverable

A public GitHub repository with five documented mini-projects, a final report, a demo notebook and video, and text ready to drop into your scholarship application.

You are done with this week when

- A stranger can understand what you built from the README in 5 minutes.
- Every result number in the README can be reproduced with a command.
- You have an SOP paragraph and CV bullets that cite real numbers.

4. Turning this into a scholarship story

Admissions committees look for three things: **depth of understanding**, **ability to finish**, and **research thinking**. This project demonstrates all three if you present it correctly.

What to emphasize

- **Built from first principles:** "I implemented backpropagation, Adam, and Transformer attention from scratch and verified them against reference implementations."
- **Rigor:** gradient checks, fixed seeds, repeated runs, ablation tables, honest limitations.
- **Reproducibility:** anyone can run it with one command.
- **Progression:** theory, then numpy, then autograd, then PyTorch, then research-style analysis. That shows a learning trajectory, not a one-off.
- **Connection to your research:** in the final section, link the skills to your existing research work (Zero Trust and guardrail evaluation), so the project reads as a foundation for graduate research.

SOP paragraph template (edit with your real numbers)

To deepen my understanding beyond coursework, I spent twelve weeks rebuilding core machine-learning components from first principles. I implemented linear and logistic regression with full evaluation metrics, a scalar autograd engine, and a neural network with SGD, momentum and Adam, reaching [X] percent accuracy on MNIST. I then studied "Attention Is All You Need", implemented multi-head attention, and trained a small Transformer language model, running ablations on [positional encoding, number of heads, residual connections, depth] and reporting the results. This process taught me [one honest lesson, for example how initialization and learning rate interact], and it prepared me for research in [your area], which I hope to continue at [University/Lab].

CV bullet templates

- Implemented autograd, MLP, Adam, and a Transformer language model from scratch (numpy and PyTorch); [X] percent on MNIST; open-source repo with tests and reproducible experiments.
- Conducted ablation study on a small Transformer (positional encoding, heads, residual connections, depth) and documented findings in a 6-page report.
- Rebuilt research pipeline components independently and validated against the original implementation.

Where to publish

- GitHub repository (main proof), pinned on your profile.
- A short blog post or a LinkedIn article on one lesson (for example "What I learned building backprop from scratch").
- Optional: a Kaggle notebook of the Transformer or MNIST work, which shows up in searches.

5. The cheat-sheet system (your fix for the memory problem)

This is the most important habit in the whole plan. Every time you look something up and use it, add a line to **cheat_sheet.md**. Record the **problem in your own words**, the **function**, and a **tiny example**. After 12 weeks you will have 100 or more entries you wrote and used yourself.

I want to...	Use	Tiny example
Reshape wide data to long	pd.melt	pd.melt(df, id_vars=['id'], value_vars=['a','b'])
Combine two tables on a key	pd.merge	pd.merge(df1, df2, on='id', how='left')
Group and aggregate	df.groupby().agg()	df.groupby('city').agg(mean_price=('price','mean'))
Add a batch of ones column	np.c_ / np.hstack	X = np.c_[np.ones(len(X)), X]
Stable softmax	subtract row max	x - x.max(axis=1, keepdims=True)
Boolean count	np.sum(mask)	np.sum((y == 1) & (pred == 1))
Make subplots	plt.subplots	fig, ax = plt.subplots(1, 2, figsize=(10,4))
Torch: switch to eval mode	model.eval() + no_grad	with torch.no_grad(): out = model(x)

Matplotlib recipes file

Also keep a **plots.py** with 5 ready templates: line plot with legend, bar chart, scatter with colors, two subplots, and heatmap. For every new project, copy a template and edit it. Nobody memorizes matplotlib; they reuse recipes.

Spaced-review calendar

- **Daily:** rewrite yesterday's key snippet from memory (10 minutes).
- **Every Day 7:** rewrite one important snippet of the week from memory.
- **Every 4th week:** pick 5 random cheat-sheet entries and use them in a small script without looking.

6. Troubleshooting and common mistakes

Problem	Likely cause	What to do
Loss is NaN or infinity	Learning rate too high, unstable softmax or log(0)	Lower the learning rate, subtract the max in softmax, clip probabilities before log
Loss does not decrease	Bug in gradient, wrong sign in update, data not standardized	Run gradient check, print gradient norms, overfit 10 samples first (should reach near-zero loss)
Shape mismatch errors	Transposed matrix, missing keepdims, wrong axis	Print every shape, write expected shapes as comments, use asserts
Train accuracy high, test low	Overfitting or data leakage	Add regularization, check that scaling used train statistics only
Transformer outputs gibberish	Mask missing or wrong, positional info missing, too few steps	Print the attention matrix, verify the causal mask, train longer, lower the learning rate
Colab session dies	Time limit or memory	Save checkpoints every few hundred steps and reload
You feel stuck for days	Task too big	Split it into 3 smaller tasks, get one thing working, then continue. Ask a senior or a forum a specific question with your minimal code.

The sanity test that saves hours

Before training on a full dataset, **overfit a tiny batch** (10 to 20 samples). A correct model and training loop must reach near-zero loss on it. If it cannot, you have a bug in the model, gradient, or loss. Fix that before anything else.

7. Suggested learning resources

Use these as background reading, not as code to copy. Pick one per topic; do not collect more than you can use.

Topic	Resource	How to use it
Autograd	Andrej Karpathy's micrograd lecture (YouTube)	Watch a segment, pause, and write your version. Do not type along blindly.
Backprop and NN basics	Stanford CS231n course notes	Read the backprop and optimization notes for the math and intuition.
Language model	Karpathy's "Let's build GPT" lecture	Use after you attempted Week 8 yourself, to compare designs.
The paper	"Attention Is All You Need" (Vaswani et al.)	Weeks 7 to 9 primary source; annotate it.
Visual intuition	Jay Alammar's "The Illustrated Transformer"	Read on Day 1 of Week 7 for the mental picture.
PyTorch	Official PyTorch tutorials (60-minute blitz)	Week 10 reference; skim, then rebuild from your own projects.
numpy / pandas	Official user guides	Look up functions after you attempt, then add to your cheat-sheet.

8. Master checklist

Done	Item
☐	Week 1: LinearRegression class (closed form + gradient descent), tests, loss curve
☐	Week 2: Logistic regression, metrics module (confusion, precision, recall, F1, ROC, PR), imbalance experiment
☐	Week 3: Value class with add, mul, sub, neg, backward, gradient tests
☐	Week 4: Full op set, Neuron/Layer/MLP, moons decision boundary
☐	Week 5: Numpy MLP with manual backprop, gradient check, about 95 percent on MNIST
☐	Week 6: SGD, momentum, Adam, regularization, LR and init studies, 97 percent or more, experiment report
☐	Week 7: Scaled dot-product, masking, multi-head attention, positional encoding
☐	Week 8: Transformer block, small character-level language model, generated samples
☐	Week 9: Four ablations, analysis plots, mini-paper
☐	Week 10: Idiomatic PyTorch pipeline, training loop memorized
☐	Week 11: Rebuilt one research component yourself, CLI training script
☐	Week 12: README, final report, demo notebook and video, SOP paragraph, CV bullets
☐	Cheat-sheet has 100 or more entries; log.md has an entry for most days

Final advice: consistency beats intensity. Ninety focused minutes a day, honest attempts before looking things up, and a habit of writing everything down will get you further than any single heroic weekend. If a week goes badly, restart on Day 1 of the next week without guilt. Good luck with the scholarship applications!